

Block Drill Exam

Twelve short blocks of TypeScript — run the card line by line, write the notes, then the verdict; answers follow each block

October 2026

How to run it. For each block: read the requirement, then go line by line with the card (Kind · Source · Prim/Obj · Ticket? · Nothing? · Who else? · Kept? → note). Write the note for every numbered line in a column. Then, reading only your notes, list the findings and give confidence + verdict. Only then uncover the answer. Budget: 8 minutes per block for 1–6, 12 minutes for 7–12. Assume `async` functions where the name suggests I/O (`getUser`, `save`, `load`, `db.query`, `fetch`).

Block 1 — Sum of paid orders

Requirement: return the total amount of orders whose status is `"paid"`, as a number; do not modify the input.

```
1 function sumPaid(orders: Order[]): number {
2   let total = 0;
3   for (const o of orders) {
4     if (o.status = "paid") {
5       total += o.amount;
6     }
7   }
8   orders.sort((a, b) => a.amount - b.amount);
9   return total;
10 }
```

Notes

Line	Note
1	declaration; declared to return a number
2	<code>total</code> holds 0, primitive, own
3	<code>o</code> holds each element, shared with <code>orders[]</code>
4	control with <code>=</code> inside: assigns <code>"paid"</code> to <code>o.status</code> , always truthy; mutates every element
5	<code>total</code> holds the running sum; <code>NaN</code> if any <code>amount</code> is not a number
8	mutating <code>sort</code> on the parameter: caller's array reordered
9	returns a number; fine on every path

Findings (ranked)

- Line 4: `=` for `===`; every order is marked paid and every amount is summed; violates the requirement and corrupts input.
- Line 8: `orders.sort(...)` mutates the caller's array; the requirement says do not modify; also the sort serves no purpose here.
- Line 5: no guard against a non-numeric `amount`; one bad value poisons `total` to `NaN` (minor; depends on `Order` typing).

Confidence 1 · Reject.

Core clause wrong on every input; input mutated.

Block 2 — Find a user by id

Requirement: GET /users/:id returns the user as JSON, 404 if not found.

```
1 app.get("/users/:id", (req, res) => {
2   const user = users.find((u) => u.id === req.params.id);
3   if (!user) res.status(404).json({ error: "not found" });
4   res.json({ id: user.id, name: user.name });
5 });
```

Notes

Line	Note
1	sync handler; req, res typed by Express
2	user holds an element of users[] or undefined; req.params.id is a string, so if u.id is a number this is always undefined
3	sends 404 but does not return; execution continues
4	user.name on undefined crashes; and this is a second response after a 404

Findings

- 1. Line 3: missing return after res.status(404).json(...); line 4 runs, crashes on user.id for a missing user, and attempts a second send.
- 2. Line 2: req.params.id is a string; if User.id is a number, === never matches and every request is 404. Convert with Number(...) or compare as strings.

Confidence 1 • Request changes.

Both findings are on the only path the requirement names.

Block 3 — Load config

Requirement: parse a JSON string into `{ port: number; debug: boolean }`; `debug` defaults to `false`; throw a clear error on invalid input.

```
1 function loadConfig(raw: string): Config {
2   const data = JSON.parse(raw) as Config;
3   const debug = data.debug || false;
4   if (data.port > 65535) throw new Error("bad port");
5   return { port: data.port, debug };
6 }
```

Notes

Line	Note
2	<code>data</code> holds whatever the string contained, labeled <code>Config</code> with no check; <code>JSON.parse</code> throws on malformed input with a parser message, not a clear one
3	<code>debug</code> holds <code>false</code> for <code>undefined</code> (fine) and also for <code>false</code> (fine); <code> </code> is acceptable here because <code>false</code> → <code>false</code>
4	<code>data.port</code> may be <code>undefined</code> , a string, or <code>NaN</code> ; <code>undefined > 65535</code> is <code>false</code> , so it passes; negative passes; <code>"abc"</code> passes
5	returns an object whose <code>port</code> may be anything

Findings

- Line 4: no check that `port` is a number, integer, or ≥ 1 ; `undefined`, `"80"`, `-1`, `NaN` all pass. Use `Number.isInteger(data.port) && data.port >= 1 && data.port <= 65535`.
- Line 2: `JSON.parse` unguarded; malformed input throws a `SyntaxError` instead of the required clear error; wrap in `try`.
- Line 2: the `as Config` is a type hole; nothing validates the shape (this is the mechanism behind finding 1).

Confidence 2 • Request changes.

Line 3 is fine; say so.

Block 4 — Fetch users in parallel

Requirement:

given ids, return the users in the same order; fail if any fetch fails.

```
1  async function getUsers(ids: string[]): Promise<User[]> {
2    const out: User[] = [];
3    ids.forEach(async (id) => {
4      const u = await getUser(id);
5      out.push(u);
6    });
7    return out;
8  }
```

Notes

Line	Note
2	<code>out</code> holds a new empty array
3	<code>forEach</code> with an async callback; <code>forEach</code> returns <code>undefined</code> and waits for nothing
4	inside the callback, <code>u</code> is awaited correctly, but nobody awaits the callback
5	pushes happen later, in completion order, after line 7 has already run
7	returns <code>out</code> while it is still empty

Findings

- Lines 3–7: `forEach` does not await its callbacks; `getUsers` resolves to `[]`. Even if it waited, pushes would land in completion order, not input order. Fix: `return Promise.all(ids.map((id) => getUser(id)))`, which also preserves order and rejects on first failure as required.

Confidence 1 • Reject.

Returns the wrong value on every input.

Block 5 — Rate limit by key

Requirement: allow at most 100 requests per key per rolling 60 s; respond 429 with `Retry-After` in seconds otherwise.

```
1  const hits = new Map<string, number[]>();
2  export function limit(req: Request, res: Response, next: NextFunction) {
3    const key = req.headers["x-api-key"] as string;
4    const now = Date.now();
5    const recent = (hits.get(key) ?? []).filter((t) => now - t < 60_000);
6    if (recent.length > 100) {
7      res.set("Retry-After", String(Math.ceil((recent[0] + 60_000 - now) / 1000)));
8      return res.status(429).end();
9    }
10   recent.push(now);
11   hits.set(key, recent);
12   next();
13 }
```

Notes

Line	Note
1	<code>hits</code> is module-level shared state; grows with keys
3	<code>key</code> holds a string OR <code>undefined</code> OR <code>string[]</code> ; the cast hides it; <code>undefined</code> becomes one shared bucket
5	<code>recent</code> holds a NEW array (filter), own; the stored array is untouched
6	boundary: <code>> 100</code> allows the 101st request
7	<code>Math.ceil</code> can yield 0 when <code>recent[0]</code> is about to expire; <code>recent[0]</code> is <code>undefined</code> only if <code>recent</code> is empty, which cannot happen here
8	returns after sending; correct
10–11	on the allowed path the pruned array is stored; on the rejected path (6–8) it is not, so stale timestamps persist until the next allowed request (minor)
12	<code>next()</code> called only on the allowed path; correct

Findings

- Line 6: `> 100` lets the 101st through; `>= 100`.
- Line 3: missing header is not rejected; all keyless callers share the `undefined` bucket; reject with 401 before use.
- Line 1: unbounded growth; any caller can invent keys; no cap, no sweep.
- Line 7: `Retry-After` can be `0`; use `Math.max(1, ...)`.
- Design note: per-process `Map`; with several instances the limit is per instance.

Positives: sliding-window log is the right structure; `filter` makes a new array so the stored one is not mutated mid-check; `return` after the 429.

Confidence 2 • Request changes.

Block 6 — Cache with TTL

Requirement: `getUser` caches results for 60 s; concurrent calls for the same id must not fetch twice.

```
1  const cache = new Map<string, { user: User; at: number }>();
2  async function getUser(id: string): Promise<User> {
3    const hit = cache.get(id);
4    if (hit && Date.now() - hit.at < 60_000) return hit.user;
5    const user = await fetchUser(id);
6    cache.set(id, { user, at: Date.now() });
7    return user;
8  }
```

Notes

Line	Note
3	<code>hit</code> holds the cache entry (shared with the map) or <code>undefined</code>
4	boundary: exactly 60 000 ms old is NOT fresh (<code><</code>); fine either way, but say which
5	yield point; <code>user</code> holds the fetched User after the await
6	writes the entry; between 3 and 6 another call for the same id sees a miss
7	returns a User that is also stored in the cache: callers share it; a caller mutating <code>user</code> mutates the cached copy

Findings

- Lines 3–6: check-then-fill across an `await`; concurrent misses all call `fetchUser`; violates the requirement. Store the Promise: `cache.set(id, fetchUser(id).then(...))` or keep an in-flight map.
- Line 7: the returned object is the cached object; any caller that mutates it corrupts the cache for everyone (design note or finding depending on callers).
- Line 1: no eviction; expired entries stay until overwritten; grows with distinct ids.

Confidence 2 • Request changes.

Happy-path correct; the named concurrency clause fails.

Block 7 — Tag index

Requirement: `add(doc)` indexes a doc under each tag; `byTag(tag)` returns docs sorted by score descending without exposing internal arrays.

```
1 class TagIndex {
2   private byTag = new Map<string, Doc[]>();
3   add(doc: Doc) {
4     for (const t of doc.tags) {
5       this.byTag.get(t).push(doc);
6     }
7   }
8   byTag(tag: string): Doc[] {
9     const list = this.byTag.get(tag) ?? [];
10    return list.sort((a, b) => a.score > b.score);
11  }
12 }
```

Notes

Line	Note
2	<code>byTag</code> field: a Map, internal state
5	<code>get(t)</code> is <code>undefined</code> for a new tag; <code>.push</code> throws on the first ever call
8	method named <code>byTag</code> shadows the field named <code>byTag</code> : <code>this.byTag</code> on line 9 is the METHOD, not the map (name collision)
9	<code>list</code> holds the internal array (shared) or a new <code>[]</code>
10	<code>sort</code> mutates the internal array and returns it: internal state exposed and reordered; comparator returns a boolean

Findings

- Line 5: `undefined.push` crash on any new tag; `(this.byTag.get(t) ?? [])` then `set`.
- Lines 2/8: field and method share the name `byTag`; TS will error or the method will shadow; rename one.
- Line 10: comparator returns a boolean, not a number; order unreliable; use `b.score - a.score`.
- Line 10: `sort` on the internal array mutates it and returns the same reference; violates “without exposing”; sort a copy: `[...list].sort(...)`.

Confidence 1 • Reject.

Cannot be used at all until 1 and 2 are fixed.

Block 8 — Retry with backoff

Requirement: call `op()` up to 3 times, waiting 200 ms between attempts; throw the last error after the third failure.

```
1  async function retry<T>(op: () => Promise<T>): Promise<T> {
2    let lastErr: unknown;
3    for (let i = 1; i < 3; i++) {
4      try {
5        return op();
6      } catch (e) {
7        lastErr = e;
8        sleep(200);
9      }
10   }
11   throw new Error(`failed: ${lastErr as Error}.message`);
12 }
```

Notes

Line	Note
3	<code>i</code> runs 1, 2: two attempts, not three
5	<code>return op()</code> hands the Promise out of the <code>try</code> un-awaited; a rejection is never caught here; the loop never retries
8	<code>sleep(200)</code> not awaited; no delay
11	<code>lastErr</code> may be anything; <code>as Error</code> can crash on <code>.message</code> if a non-Error was thrown; reached only if <code>op</code> throws synchronously

Findings

- Line 5: missing `await`; the `catch` never sees a rejection, so there is exactly one attempt and the function returns the first Promise, success or failure. `return await op()`.
- Line 3: `i < 3` from 1 is two iterations; `i <= 3`.
- Line 8: `sleep` not awaited; retries are immediate.
- Line 11: `(lastErr as Error).message` unsafe; narrow with `instanceof`.

Confidence 1 · Reject.

Every clause of the requirement fails.

Block 9 — Merge options

Requirement: return `defaults` overridden by `opts`; neither input may be modified; nested `headers` must merge, not replace.

```
1  const DEFAULTS = { retries: 3, headers: { "Accept": "json" } };
2  function merge(opts: Partial<typeof DEFAULTS>) {
3    const out = { ...opts, ...DEFAULTS };
4    out.headers = Object.assign(DEFAULTS.headers, opts.headers);
5    return out;
6  }
```

Notes

Line	Note
1	<code>DEFAULTS</code> is module-level shared state; <code>DEFAULTS.headers</code> is an object inside it
3	spread order reversed: <code>DEFAULTS</code> wins over <code>opts</code> ; <code>out</code> is a new outer object; <code>out.headers</code> shared with <code>DEFAULTS.headers</code>
4	<code>Object.assign(target, src)</code> MUTATES its first argument: <code>DEFAULTS.headers</code> is permanently changed for every later caller; <code>out.headers</code> is that same object
5	returns <code>out</code> , whose <code>headers</code> is the shared, now-mutated defaults object

Findings

- 1. Line 4: `Object.assign(DEFAULTS.headers, ...)` mutates the module-level defaults; every subsequent call sees the previous caller's headers. Use `{ ...DEFAULTS.headers, ...opts.headers }`.
- 2. Line 3: spread order reversed; `opts.retries` is discarded. `{ ...DEFAULTS, ...opts }`.
- 3. Line 5: the returned object shares `headers` with `DEFAULTS` even after fixing 2 unless 1 is also fixed.

Confidence 1 • Reject.

Both clauses fail and shared state is corrupted.

Block 10 — Paginated list

Requirement: `page` ≥ 1 (default 1), `size` 1–50 (default 20), else 400; return `{ items, total }` from the DB.

```
1 app.get("/items", async (req, res) => {
2   const page = Number(req.query.page) || 1;
3   const size = Number(req.query.size) || 20;
4   if (size > 50) return res.status(400).json({ error: "size" });
5   const offset = page * size;
6   const items = await db.query("SELECT * FROM items LIMIT $1 OFFSET $2", [size, offset]);
7   const total = db.query("SELECT COUNT(*) FROM items");
8   res.json({ items, total });
9 });
```

Notes

Line	Note
2	<code>page</code> holds a number; <code>0</code> , <code>NaN</code> , negative-after- <code> </code> : <code>-5</code> passes as <code>-5</code>
3	<code>size</code> holds a number; <code>0</code> $\rightarrow$ 20 silently; <code>-1</code> passes; <code>NaN</code> $\rightarrow$ 20
4	only the upper bound is checked
5	<code>offset</code> for page 1 is <code>size</code> , skipping the first page; should be <code>(page - 1) * size</code>
6	awaited; parameterized; fine
7	NOT awaited; <code>total</code> holds a Promise
8	sends <code>{ items, total: {} }</code> (a Promise serializes to <code>{}</code>)

Findings

- Line 7: missing `await`; `total` is a Promise and serializes as `{}`.
- Line 5: off-by-one; page 1 skips the first `size` items.
- Lines 2–4: `page < 1`, `size < 1`, and `NaN` are not rejected; `size=0` silently becomes 20. Validate with `Number.isInteger` and both bounds.
- Line 1: async handler in Express 4: a thrown error hangs the request unless wrapped (design note).

Positives: parameterized SQL on line 6; `return` after the 400.

Confidence 2 • Request changes.

Block 11 — Event bus

Requirement: `on` registers; `emit` calls every handler registered at emit time, waits for async handlers, and rejects with the first error after all have run.

```
1 class Bus {
2   private handlers: Record<string, Handler[]> = {};
3   on(evt: string, h: Handler) { (this.handlers[evt] || []).push(h); }
4   async emit(evt: string, payload: unknown) {
5     const errors: unknown[] = [];
6     for (const h of this.handlers[evt] ?? []) {
7       try { h(payload); } catch (e) { errors.push(e); }
8     }
9     if (errors.length) throw errors[0];
10  }
11 }
```

Notes

Line	Note
2	plain object keyed by user-supplied event names; prototype keys reachable
3	<code>evt = "constructor": this.handlers["constructor"]</code> is a function (inherited), truthy, so <code> =</code> does not assign, and <code>.push</code> throws
6	iterates the live array; a handler that calls <code>on</code> during emit changes the array mid-loop (requirement says "registered at emit time")
7	<code>h(payload)</code> not awaited; an async handler's rejection escapes the <code>try</code> ; <code>emit</code> resolves before async handlers finish
9	correct shape: throws the first after the loop

Findings

- Line 7: handlers are not awaited; async failures are unhandled and `emit` does not wait; `await h(payload)` (or collect Promises and `allSettled`).
- Line 2–3: `Record<string, ...>` with user keys; `"constructor"` or `"__proto__"` as an event name breaks or pollutes; use a `Map`.
- Line 6: iterate a snapshot, `[...list]`, so registration during emit does not affect this emit.

Confidence 2 · Request changes.

Structure is right; the async clause fails.

Block 12 — The clean one

Requirement: debounce: return a function that calls `fn` with the latest args once `ms` have passed with no further calls; expose `cancel()`.

```
1 function debounce<A extends unknown[]>(fn: (...a: A) => void, ms: number) {
2   let timer: ReturnType<typeof setTimeout> | null = null;
3   let last: A | null = null;
4   const run = (...args: A) => {
5     last = args;
6     if (timer !== null) clearTimeout(timer);
7     timer = setTimeout(() => {
8       timer = null;
9       if (last !== null) fn(...last);
10    }, ms);
11  };
12  run.cancel = () => { if (timer !== null) clearTimeout(timer); timer = null; last = null; };
13  return run;
14 }
```

Notes

Line	Note
2–3	two closure variables, initialized to <code>null</code> ; checked with <code>!== null</code> , not truthiness (so a timer id of 0 would be handled)
5	<code>last</code> holds the latest args array (shared with the caller's arguments; fine)
6	every call clears the pending timer: this is debounce, not throttle
7–10	timer set; callback nulls the timer BEFORE calling <code>fn</code> , so a throwing <code>fn</code> does not leave the debouncer stuck
12	<code>cancel</code> clears and resets both; assigning a property onto an arrow is a TS type complaint unless the return type declares it (nit)
13	returns the function

Findings

- Nit: `run.cancel = ...` on an arrow needs a declared return type (`run` as `typeof run & { cancel(): void }`) or an object form; compiles with a type error under strict settings.
- Nit: `timer !== null` is correct and deliberate; note it as a positive, not a finding.

Positives: clears on every call (true debounce); nulls the timer before calling `fn`; `cancel` resets both variables; generic preserves argument types; null checks are explicit rather than truthy.

Confidence 5 • Approve.

If you listed more than the one typing nit, you invented findings.

Scoring yourself

Block	Planted	You should have found
1	3	the <code>=</code> , the <code>sort</code>
2	2	the missing <code>return</code> , the string/number id
3	3	the port validation, the unguarded parse
4	1	<code>forEach</code> async
5	4 + design	<code>> 100</code> , missing key, growth
6	3	check-then-fill race
7	4	<code>get().push</code> , the name collision, the boolean comparator, the exposed sort
8	4	missing <code>await</code> on <code>op()</code> , two attempts, <code>sleep</code> not awaited
9	3	<code>Object.assign</code> on defaults, spread order
10	3 + design	missing <code>await</code> on <code>total</code> , offset off-by-one, input bounds
11	3	handlers not awaited, plain-object keys
12	0	nothing; approve

For every miss: name the line, name the Q that would have caught it, say both out loud once. That is the whole correction.